The Book

This book teaches a small language: how to write in it, and what what we write comes to. It assumes nothing.

The language is made of a few **forms**. Each form is shown once, as a **shape**: a line with letters, where each letter marks a place that any piece of one kind can fill. Examples come with each shape, most about the game rock, paper, scissors. The shapes add up to a **map** of the language. Since any piece can go wherever its letter is, the map also shows how the pieces combine, and the sections show those combinations, each with an example.

The language has two parts. The first is for **saying**: things, functions that turn things into other things, and kinds of things. The second is for **doing**: keeping a score that changes, giving up, trying every possibility, taking turns, talking to the world outside.

1. Things

The simplest things are things that are just themselves. A kind of thing is given by listing every thing of that kind:

```
Hand = rock | paper | scissors
Bool = false | true
```

`|` reads "or". Each line **declares** a **type**: a kind of thing. A `Hand` is `rock`, `paper` or `scissors`, and nothing else. `rock`, `paper` and `scissors` are **constructors**: names that stand only for themselves.

Every declaration has this shape:

```
T = d | …                  a type, with its constructors
d ::= c                    a constructor, holding nothing
```

In a shape, a letter marks a place: `T` for any type's name, `c` for any constructor, `d` for a constructor as it is declared. `::=` reads "is one of", and `| …` reads "and more like the one before, as many as needed".

```
names    each type is declared once, and each constructor belongs to one type
```

Anything we write that says something is an **expression**. A constructor is an expression with nothing left to work out in it: a **value**. Using `e` for any expression and `v` for any value:

```
e ::= c                    a constructor
v ::= c                    a constructor
```

2. Functions

Cases

A **function** turns things into other things. It is written as **cases**: for each thing it can be given, what it gives back.

```
answer = (rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock
```

`↦` reads "gives": rock gives paper, or paper gives scissors, or scissors gives rock. Each hand is given the hand that beats it. In each case, the part in parentheses is its **pattern**, what the case is for, and the part

after `↦` is its **body**, what the case gives back. `answer = …` **defines** the name `answer`: from now on, `answer` stands for this function.

A function is used by **applying** it to an **input**, written after it in parentheses. Applying takes one **step**: the application **becomes** the body of the first case whose pattern the input **matches**. A constructor matches only itself:

```
answer(rock)
    becomes   paper
```

`paper` is a value, so `paper` is the answer. One more:

```
not = (false) ↦ true | (true) ↦ false

not(true)
    becomes   false
```

Placeholders

A pattern can also be a **placeholder**: a name that matches anything. Cases are tried in order, so a last case with a placeholder means "otherwise":

```
isRock = (rock) ↦ true | (a) ↦ false

isRock(paper)
    becomes   false
```

In the body, a placeholder stands for what it matched, and applying puts the input wherever the placeholder is:

```
beaten = (a) ↦ answer(answer(a))

beaten(rock)
    becomes   answer(answer(rock))
```

`beaten` gives the hand that a hand beats: the answer to its answer.

Several inputs

A case can have several patterns, separated by commas. The function then takes that many inputs, and they match a case when each matches its own pattern:

```
beats = (rock, scissors) ↦ true | (scissors, paper) ↦ true | (paper, rock) ↦ true | (a, b) ↦ false

beats(paper, rock)
    becomes   true
beats(rock, paper)
    becomes   false
```

Rock beats scissors, scissors beat paper, paper beats rock; otherwise, the first hand doesn't win. In the second, only the last case matches. All the cases of one function have the same number of patterns, and the function is always given that many inputs: `beats(rock)` is not an expression. The placeholders of one case are all different: in `(a, a)`, we couldn't tell which input `a` is.

Working out

`beaten(rock)` became `answer(answer(rock))`, which still has something left to work out. An input is worked out before it goes in, so the inner application goes first:

```
beaten(rock)
    becomes   answer(answer(rock))
    becomes   answer(paper)
    becomes   scissors
```

"Comes to" means "becomes, in one step or more": `beaten(rock)` comes to `scissors`. Several inputs are worked out one at a time, from left to right:

```
and = (true, b) ↦ b | (false, b) ↦ false

and(beats(paper, rock), isRock(paper))
    becomes   and(true, isRock(paper))
    becomes   and(true, false)
    becomes   false
```

So there is never more than one next step. Inside a function, nothing happens until it is applied, and then only the body of the case used is worked out. A function can also take no inputs, and is then applied to none. It is a question kept for later:

```
later = () ↦ answer(rock)

later()
    becomes   answer(rock)
    becomes   paper
```

Functions are values

A defined name can also be written on its own. It stands for its function, and it is a value: nothing in it is worked out until it is applied. So a function can be given to a function:

```
twice = (f, a) ↦ f(f(a))

twice(answer, rock)
    becomes   answer(answer(rock))
    comes to  scissors
```

What is applied here is the placeholder `f`. What is applied can be any expression. It is worked out first, then the inputs.

A function can also be **written out**, without a name, wherever an expression can go:

```
((rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock)(rock)
    becomes   paper
```

Parentheses only group: here they keep the function in one piece. Without them, `(rock)` would be applied to the last body, `rock`, since a body reaches as far as it can.

Written out inside a body, a function can use the placeholders around it. This is how a function gives back a function made from its input:

```
against = (a) ↦ (b) ↦ beats(b, a)

against(rock)
    becomes   (b) ↦ beats(b, rock)
against(rock)(paper)
    comes to  true
```

`against(rock)` is a test: does a hand beat rock? The input went wherever `a` was, even inside the inner function, where nothing is worked out yet. A function can also take a function and give back a function:

```
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(answer)
    becomes   (a) ↦ answer(answer(a))
```

`twofold(answer)` is the same function as `beaten`, made without a definition of its own. When an inner placeholder has the same name as an outer one, the name means the nearer one:

```
((a) ↦ (a) ↦ a)(rock)(paper)
    comes to  paper
```

Using itself

A definition can use any defined name: one defined before it, as `beaten` uses `answer`; one defined after it, so `beaten` could just as well come first; or its own. Nothing stops a definition from using itself forever:

```
loop = (a) ↦ loop(a)

loop(rock)
    becomes   loop(rock), and so on, forever
```

Getting stuck

```
answer(true)              stuck: no case of answer matches true
twice(rock, rock)         becomes rock(rock(rock)), stuck: rock is not a function
twice(beats, rock)        becomes beats(beats(rock)), stuck: beats takes two inputs
```

In each, something is left to work out, but a value is where it doesn't fit, so no step applies. Written directly, `rock(rock)` and `beats(rock)` are not expressions; through a placeholder, only working out finds them.

So carrying out ends in one of three ways: in a value, which is the answer; stuck; or never.

On the map

Using `n` for a defined name, `p` for a pattern and `x` for a placeholder, with `p, …` meaning any number of patterns, even none, and likewise for other letters:

```
n = (p, …) ↦ e | …          a definition: n stands for a function, with its cases

p ::= c                     matches only itself
      x                     a placeholder: matches anything

e ::= x                     a placeholder
      n                     a defined name
      (p, …) ↦ e | …        a function written out
      e(e, …)               an application

v ::= n                     a defined name
      (p, …) ↦ e | …        a function written out

((p, …) ↦ e | …)(v, …)      becomes   the body of the first case whose patterns v, … match,
                                      with what each placeholder matched in its place
n(v, …)                     becomes   the same, with the cases of n's definition

names    each defined name is defined once
names    a definition can use any defined name, its own included, even one defined after it
names    in a pattern, a name is a constructor if it was declared as one, and otherwise a placeholder
names    the placeholders of one case are all different, and none has the name of anything declared or
      defined
```

```
names    a placeholder means something in the body of its own case, functions inside it included
names    in an expression, a name means the nearest placeholder of that name; with none, it must be
      declared or defined
names    the cases of one function all have the same number of patterns
names    a defined name written right before inputs is given as many as its function takes
names    a constructor that holds nothing is written without parentheses
order    in e(e, …), first what is applied, until it is a value; then the inputs, from left to right
order    nothing happens inside a function until it is applied
writing  parentheses only group
writing  a body reaches as far as it can
ends     in a value
ends     stuck: a value is where it doesn't fit, so something is left to work out but no step applies
ends     never: carrying out goes on forever
```

How the forms combine:

```
what is applied can be     a defined name             answer(rock)
                           a placeholder              f(f(a)), in twice
                           an application             against(rock)(paper)
                           a function written out     ((a) ↦ (a) ↦ a)(rock)
an input can be            an application             answer(answer(rock))
                           a function                 twice(answer, rock)
a body can be              an application             (a) ↦ answer(answer(a))
                           a function                 (a) ↦ (b) ↦ beats(b, a)
```

3. Holding

Contents

A round of the game is two hands, one for each player. A constructor can **hold** things, its **contents**, of the types written after it:

```
Round = round(Hand, Hand)
```

A constructor that holds things is always written with them, as many as it holds:

```
round(rock, paper)
```

This is not an application: `round` is not a function, and nothing happens. It is a value: a round in which the first player showed rock and the second paper. Values built from constructors are **data**. Contents are worked out before they go in, from left to right:

```
round(rock, answer(rock))
    becomes   round(rock, paper)
```

Taking data apart

A pattern can be a constructor with patterns for its contents:

```
won = (round(a, b)) ↦ beats(a, b)

won(round(paper, rock))
    becomes   beats(paper, rock)
    becomes   true
```

`round(a, b)` matches any round, and gives `a` the first hand and `b` the second. The contents of a pattern are patterns, so a pattern can look more than one layer in:

```
draw = (round(rock, rock)) ↦ true | (round(paper, paper)) ↦ true | (round(scissors, scissors)) ↦ true |
      (other) ↦ false

draw(round(paper, rock))
    becomes   false
```

`round(paper, rock)` doesn't match `round(paper, paper)`, since `rock` doesn't match `paper`; only the last case matches it.

Holding itself

A constructor can hold things of its own type:

```
Nat = zero | succ(Nat)
```

A `Nat` is `zero`, or `succ` holding a `Nat`; `succ` reads "one more than". So `zero`, `succ(zero)`, `succ(succ(zero))` are zero, one, two, and so on, without end. A pattern sees only as many layers as it is written with, so a function that works through a `Nat` of any size uses itself, once for each layer:

```
add = (zero, b) ↦ b | (succ(a), b) ↦ succ(add(a, b))

add(succ(succ(zero)), succ(zero))
    becomes   succ(add(succ(zero), succ(zero)))
    becomes   succ(succ(add(zero, succ(zero))))
    becomes   succ(succ(succ(zero)))
```

Each step takes a layer off the first input and puts it outside, until `zero` ends it. Two and one make three. Two definitions can also use each other:

```
even = (zero) ↦ true | (succ(a)) ↦ odd(a)
odd = (zero) ↦ false | (succ(a)) ↦ even(a)

even(succ(succ(zero)))
    comes to  true
```

Declarations work the same way: a type can hold any type, its own included, even one declared after it. `Round` could just as well be declared before `Hand`.

Holding functions

A player plays by a rule: given the hand the other player showed last, it gives the hand to show next. To say what a player holds, we need the type of such rules:

```
Player = player((Hand) → Hand)
```

`(Hand) → Hand` reads "from `Hand` to `Hand`". It is a **function type**: the type of the functions that, given a hand, give back a hand.

```
next = (player(f), a) ↦ f(a)

next(player(answer), rock)
    becomes   answer(rock)
    becomes   paper
```

No pattern looks inside a function: a function can only be applied. And data is not a function: `round(rock, paper)(rock)` is stuck.

What a type means

A type says which values are of its kind:

- A value is of type `T` when it is built by one of `T`'s constructors, holding values of the types declared for its contents. `rock` is a `Hand`, and `round(rock, paper)` is a `Round`. `round(true, false)` is data too, just not a `Round`. A type defines a kind, and forbids nothing.

- A value is of type `(A, …) → B` when, applied to values of types `A, …`, it never gets stuck, and if it ends, it ends in a value of type `B`. `answer` and `beaten` are `(Hand) → Hand`. `onlyRock = (rock) ↦ paper` is not: given `paper`, it gets stuck.
- A type that holds itself is read over and over: `succ(succ(zero))` is a `Nat` because `succ(zero)` is, and `succ(zero)` is because `zero` is. A value is of such a type when following the definition, as far as it can be followed, never finds anything that does not fit.

A type can also hold itself in what a function gives back. A game is either over, saying whether you won, or it is your turn, and given your hand, it gives the rest of the game:

```
Game = over(Bool) | yourTurn((Hand) → Game)
```

`yourTurn((a) ↦ over(beats(a, rock)))` is a game of one round against rock. But a type cannot hold itself in what a function takes, as in `Test = test((Test) → Bool)`. Whether a function is a `(Test) → Bool` would depend on which values are `Test`s, which is what is being defined: the definition would go in a circle, and say nothing.

That a value is of a type can be true or false: it is a **claim**. Claims can be checked, but nothing in working out uses types, and this book leaves claims out.

On the map

Using `A` and `B` for types:

```
d ::= c(A, …)              a constructor holding contents of types A, …
A ::= T                    a type, by its name
      (A, …) → B           a function type

e ::= c(e, …)              data: a constructor with its contents
v ::= c(v, …)              data: a constructor holding values
p ::= c(p, …)              matches data built by c, with contents matching p, …

names    in a declaration, every name in a type is a declared type: any, its own included, even one
      declared after it
names    a constructor is written with as many contents as it holds, in an expression or a pattern
order    in c(e, …), the contents are worked out from left to right
types    a value is of type T when it is built by one of T's constructors, holding values of the types
         declared for its contents
types    a value is of type (A, …) → B when, applied to values of types A, …, it never gets stuck,
         and if it ends, it ends in a value of type B
types    a type that holds itself: a value is of the type when following the definition, as far as it
         can be followed, never finds anything that does not fit
types    a type can hold itself in its contents, and in what a function gives back, but not in what a
         function takes
```

How the forms combine. Every value is data or a function, so every input, every result and all contents are one or the other:

```
(Hand) → Bool                     takes data, gives data              isRock
(Hand) → (Hand) → Bool            takes data, gives a function        against
((Hand) → Hand, Hand) → Hand      takes a function, gives data        twice
((Hand) → Hand) → (Hand) → Hand   takes a function, gives a function  twofold
round(Hand, Hand)                 data holding data                   round(rock, paper)
player((Hand) → Hand)             data holding a function             player(answer)
succ(Nat)                         data holding its own kind           succ(succ(zero))
```

4. Types left open

A list of hands and a list of rounds are built the same way; only the type of the items changes. A type can leave that open, with **placeholders for types**, listed after its name:

```
List(X) = empty | cons(X, List(X))
Pair(X, Y) = pair(X, Y)
```

A `List(Hand)` is `empty`, or `cons` holding a hand and a `List(Hand)`; `cons` reads "followed by". A `List(Round)` is a list of rounds, and a `Pair(Bool, Nat)` holds a `Bool` and a `Nat`.

Functions need nothing like this. A function looks only at how a value is built, so one that works through a list works through a list of anything:

```
each = (f, empty) ↦ empty | (f, cons(a, rest)) ↦ cons(f(a), each(f, rest))

each(answer, cons(rock, cons(paper, empty)))
    comes to  cons(paper, cons(scissors, empty))
each(isRock, cons(rock, empty))
    comes to  cons(true, empty)
```

Counting the rounds the first player won:

```
more = (true, total) ↦ succ(total) | (false, total) ↦ total
wins = (empty) ↦ zero | (cons(this, rest)) ↦ more(won(this), wins(rest))

wins(cons(round(paper, rock), cons(round(rock, paper), empty)))
    comes to  more(true, wins(cons(round(rock, paper), empty)))
    comes to  more(true, more(false, zero))
    comes to  succ(zero)
```

On the map

Using `X` for a placeholder for a type:

```
T(X, …) = d | …            a type that takes types; T = d | … is this, taking none
A ::= T(A, …)              a type, filled with types
      X                    a placeholder for a type

names    a declaration's own placeholders can be used in its types
names    the placeholders of one declaration are all different, and none has the name of a declared type
names    a type is filled with as many types as it takes
types    a value is of type T(A, …) when it is built by one of T's constructors, holding values of the
         types declared for its contents, with A, … in place of T's placeholders
```

How the forms combine. The types in `T(A, …)` are any types, so types nest: `List(List(Hand))`, `List((Hand) → Hand)`, `Pair(Round, List(Bool))`.

The core

That is the whole of saying. Every value is data or a function, and each is made one way and used one way:

	made by	**used by**
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs

A placeholder stands for what a case was given; a defined name stands for its function, which may use the name itself. A type says which values are of a kind. Nothing changes: an expression and what it

comes to say the same thing, the second more directly. The rest of the book is about doing.

5. A memory, written out

As they play, the players keep score. A score changes, and in the core nothing changes. But a change can be said with two values, the score before and the score after:

```
score = (this, total) ↦ more(won(this), total)

score(round(rock, paper), score(round(paper, rock), zero))
    comes to  score(round(rock, paper), succ(zero))
    comes to  succ(zero)
```

The score is a **memory**: something kept, and changed as we go. Here it is written out as a value, handed from one function to the next.

That has a cost. This asks whether the first player won both of two rounds:

```
both = (first, second) ↦ and(won(first), won(second))
```

To keep the score as well, each round must give back two things, whether it was won and the new score:

```
play = (this, total) ↦ pair(won(this), more(won(this), total))
```

and `both` must take the score, hand it through each round in turn, and give it back with its answer. When this book defines a name again, the new definition replaces the old:

```
both = (first, second, total) ↦ ((pair(a, m)) ↦ ((pair(b, t)) ↦ pair(and(a, b), t))(play(second, m)))
      (play(first, total))

both(round(paper, rock), round(rock, paper), zero)
    comes to  pair(false, succ(zero))
```

This reads: play the first round with the score `total`, and call what it gives back `a` and `m`; play the second with `m`, and call what that gives back `b` and `t`; give back `and(a, b)`, with the score `t`. `both` doesn't care about the score at all: it only hands it along. So must every function that uses `both`, and every function that uses those. One small idea, a score, touches everything on the way.

What we want is to mention the score only where it is used.

6. Requests and handlers

Take a simpler case of the same problem first. To know whether a hand wins, a function needs the hand the opponent plays. Instead of being handed it, the function can ask for it:

```
effect Opponent = opponent : () → Hand
```

This declares an **effect**, `Opponent`, with one **request**, `opponent`. Like a function type, `() → Hand` says what the request takes, nothing, and what it is answered with, a `Hand`. A request is written like an application, with as many inputs as it is declared with:

```
beatsOpponent = (a) ↦ beats(a, opponent())
```

Working out `beatsOpponent(rock)` goes this far:

```
beatsOpponent(rock)
    becomes   beats(rock, opponent())
```

A request is not a value, and it takes no step of its own: it waits to be answered from outside the expression.

A **handler** answers it. It goes around the expression that asks:

```
handle beatsOpponent(rock) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)
```

A handler has cases, like a function. The first is for the value the expression inside comes to: here it gives that value back. The second is for the request `opponent()`, and it answers `scissors` by giving it to `k`.

`k` is **the rest**: what was left to do when the request was made, written as a function. `beats(rock, opponent())` is waiting for a hand to put where `opponent()` is; given one, the rest puts it there and goes on, inside the same handler:

```
(hand) ↦ handle beats(rock, hand) with …
```

Here, and from now on, `with …` stands for the same handler's cases, written again. When the next thing to work out is the request, the handler takes a step: it becomes the body of its case for the request, with the rest in place of `k`:

```
handle beatsOpponent(rock) with …
    becomes   handle beats(rock, opponent()) with …
    becomes   ((hand) ↦ handle beats(rock, hand) with …)(scissors)
    becomes   handle beats(rock, scissors) with …
    becomes   handle true with …
    becomes   true
```

In the last step, the expression inside is a value, so the handler becomes the body of its case for values.

`beatsOpponent` never says where the opponent's hand comes from: the handler does. A handler that answered `paper` would make the same `beatsOpponent(rock)` come to `false`, without changing `beatsOpponent`.

A request that a handler has no case for passes it by, to a handler further out:

```
handle (handle beatsOpponent(rock) with (outcome) ↦ not(outcome)) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)

    comes to  false
```

The inner handler has no case for `opponent()`, so the outer one answers it. The rest goes on inside both handlers, so the inner one still receives `true`, and turns it into `false`.

A request that no handler around it has a case for is **waiting**: carrying out ends there, waiting for an answer. `beatsOpponent(rock)` on its own, with no handler, comes to `beats(rock, opponent())` and waits.

On the map

Using `F` for an effect's name, `r` for a request and `h` for a handler's case:

```
effect F = r : (A, …) → B | …    an effect, with its requests: what each takes, and what it is answered
      with

e ::= r(e, …)                    a request
      handle e with h | …        a handler around e

h ::= (p) ↦ e                    for the value the expression inside comes to
      (r(p, …), x) ↦ e           for a request: patterns for its inputs, and a placeholder for the rest
```

```
handle v with h | …     becomes   the body of the first case for values that v matches,
                                  with what each placeholder matched in its place
handle e with h | …     becomes   the body of the first case for r whose patterns v, … match,
                                  with what they matched in place, and the rest in place of x,
                                  when the next thing to work out in e is a request r(v, …),
                                  and no handler in e around that request has a case for r

rest     the rest is (x) ↦ handle e' with h | …, where e' is e with x where the request was,
         and x is a placeholder used nowhere in e or in the handler's cases
names    each effect is declared once, and each request belongs to one effect
names    a request is written with as many inputs as it is declared with, in an expression or a
      handler's case
order    in r(e, …), the inputs are worked out from left to right
order    in handle e with h | …, steps happen in e
ends     waiting: the next thing to work out is a request that no handler around it has a case for
```

How the forms combine:

```
a request can be inside any expression     beats(a, opponent())
a handler can be inside a handler          handle (handle beatsOpponent(rock) with …) with …
a request inside a function waits          beatsOpponent asks only when it is applied
```

Requests and handlers are made one way and used one way, like data and functions:

	made by	used by
effect	a request	a handler, which answers it

7. Ways to go on

The handler above went on once, with one answer. But it holds the rest as a value, `k`, and can do with it whatever can be done with a function.

It can never go on:

```
handle beatsOpponent(rock) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ false

    comes to  false
```

The case never applies `k`, so the rest never happens: the handler's answer stands in for the whole. This is how errors work. Something asks for what can't be given, and the handler decides what happens instead.

It can go on several times. This handler asks, for each hand the opponent could play, whether `rock` wins, and counts the wins:

```
count = (true) ↦ succ(zero) | (false) ↦ zero

handle beatsOpponent(rock) with
      (outcome) ↦ count(outcome)
    | (opponent(), k) ↦ add(k(rock), add(k(paper), k(scissors)))

    comes to  succ(zero)
```

Each `k(…)` goes on from the request with a different hand, and comes to a number through the handler's first case: `k(rock)` and `k(paper)` come to `zero`, and `k(scissors)` to `succ(zero)`. Against the three hands, rock wins once. This is search: one question, every answer tried.

8. A memory, said once

An effect can take placeholders for types, as a type can:

```
Unit = unit
effect State(X) = get : () → X | put : (X) → Unit
```

`State(Nat)` is a memory of a number. `get` asks what it holds, and `put` sets it. `put` has nothing to give back, so it is answered with `unit`, the only value of `Unit`, which says nothing.

To do one thing and then another:

```
then = (u, a) ↦ a
```

Inputs are worked out from left to right, so `then` works out its first input, then its second, and gives back the second. Recording a round:

```
record = (true) ↦ then(put(succ(get())), true) | (false) ↦ false
```

Given that a round was won, `record` adds one to the score and gives back `true`; given that it was lost, it changes nothing. And `both`, once more:

```
both = (first, second) ↦ and(record(won(first)), record(won(second)))
```

Only `record`, the one place that changes the score, mentions it. Nothing is handed along, and a function that uses `both` doesn't change at all.

A handler holds the memory:

```
keep = (task, start) ↦ (handle task() with
                            (outcome) ↦ ((m) ↦ pair(outcome, m))
                          | (get(), k) ↦ ((m) ↦ k(m)(m))
                          | (put(b), k) ↦ ((m) ↦ k(unit)(b)))(start)
```

`keep` takes a task, kept for later as a function with no inputs, and a starting value. Given directly, the task would be worked out before it went in, and its requests made before the handler was around them; kept for later, it starts at `task()`, inside the handler. Each case of the handler gives back a function waiting for the memory so far, `m`, and `(start)` gives it the first one. The cases read:

- when the task comes to a value: given `m`, give back that value together with `m`;
- when asked for the memory: given `m`, answer `m`, and go on with the memory still `m`;
- when told to set it to `b`: given `m`, answer `unit`, and go on with the memory `b`.

The rest, `k`, has the handler around it, so it too gives back a function waiting for the memory. That is why what `k(m)` gives back is given the memory in turn: `k(m)(m)`.

Two rounds, the first won and the second lost, starting from zero. `k` is the rest after the first request, `get()`, and `k'` the rest after the second, `put(succ(zero))`:

```
keep(() ↦ both(round(paper, rock), round(rock, paper)), zero)
    comes to  ((m) ↦ k(m)(m))(zero)                    the first request is get()
    becomes   k(zero)(zero)
    comes to  ((m) ↦ k'(unit)(succ(zero)))(zero)       the next is put(succ(zero))
    becomes   k'(unit)(succ(zero))
    comes to  ((m) ↦ pair(false, m))(succ(zero))       the rest comes to false
    becomes   pair(false, succ(zero))
```

Not both rounds were won, and the score is one. The memory is handed along exactly as in section 5, but that is written once, in the handler, instead of in every function on the way.

In the core, the order in which inputs were worked out never changed what they came to. With requests it does. Asked after it is set, the memory is one; asked before, it is zero:

```
keep(() ↦ then(put(succ(zero)), get()), zero)                 comes to  pair(succ(zero), succ(zero))
keep(() ↦ ((s) ↦ then(put(succ(zero)), s))(get()), zero)       comes to  pair(zero, succ(zero))
```

In the second, `get()` is the input of the function written out, so it is worked out first, before `put`.

`keep` never looks at the memory, only hands it on, so it keeps a `State(Hand)` as well as a `State(Nat)`:

```
keep(() ↦ then(put(paper), get()), rock)                      comes to  pair(paper, paper)
```

On the map:

```
effect F(X, …) = r : (A, …) → B | …     an effect that takes types; effect F = … is this, taking none
```

9. Many memories

`keep` holds one memory. A score for each player needs two, and a program may not know in advance how many it needs. Two `keep`s, one around the other, would not do: both answer `get` and `put`, so the inner one would answer every request.

So each memory gets a name of its own, a number given when it is made: its **place**. The first memory is at `zero`, the next at `succ(zero)`, and so on.

```
effect Memory(X) = new : (X) → Nat | look : (Nat) → X | set : (Nat, X) → Unit
```

`new(a)` makes a memory holding `a`, and is answered with its place. `look(place)` asks what the memory at `place` holds, and `set(place, a)` makes it hold `a` instead.

The handler keeps every memory in one list, each at the position of its place. Four functions work with such a list: how many items it has, the item at a position, the list with the item at a position replaced, and the list with one more item at the end:

```
size = (empty) ↦ zero | (cons(a, rest)) ↦ succ(size(rest))
at = (cons(a, rest), zero) ↦ a | (cons(a, rest), succ(i)) ↦ at(rest, i)
replace = (cons(a, rest), zero, b) ↦ cons(b, rest) | (cons(a, rest), succ(i), b) ↦ cons(a, replace(rest,
      i, b))
append = (empty, b) ↦ cons(b, empty) | (cons(a, rest), b) ↦ cons(a, append(rest, b))
```

The handler is `keep` again, with the list of every memory, `t`, where `keep` had one memory, `m`. It starts with none:

```
keepAll = (task) ↦ (handle task() with
                        (outcome) ↦ ((t) ↦ outcome)
                      | (new(a), k) ↦ ((t) ↦ k(size(t))(append(t, a)))
                      | (look(place), k) ↦ ((t) ↦ k(at(t, place))(t))
                      | (set(place, a), k) ↦ ((t) ↦ k(unit)(replace(t, place, a))))(empty)
```

A new memory is answered with the next place, which is how many there are so far, and added at the end. Each player can now keep a score. `point` adds one to the memory at a place when a round was won, and `tally` scores a round for both players:

```
point = (true, place) ↦ set(place, succ(look(place))) | (false, place) ↦ unit
tally = (round(a, b), mine, yours) ↦ then(point(beats(a, b), mine), point(beats(b, a), yours))

keepAll(() ↦ ((mine, yours) ↦ then(tally(round(paper, rock), mine, yours), pair(look(mine),
      look(yours))))(new(zero), new(zero)))
    comes to  pair(succ(zero), zero)
```

The two `new(zero)` make two memories, at `zero` and at `succ(zero)`, and the function they are given calls them `mine` and `yours`. Paper beats rock, so `mine` goes up by one.

A place is a value like any other, so two placeholders can hold the same place, and then they name the same memory. Setting it through one changes what the other finds:

```
keepAll(() ↦ ((mine) ↦ ((ours) ↦ then(set(ours, succ(zero)), look(mine)))(mine))(new(zero)))
    comes to  succ(zero)
```

This is how the names of a language like Python work. Each variable is a memory at a place of its own, so `score = score + 1` is `set(score, succ(look(score)))`. And for a list `a`, `b = a` puts the same place in `b`: a change made through `b` is seen through `a`.

10. Going on later

A handler holds the rest as a value, so it can also keep it in data, and go on later.

A player can be a task of its own, which gives its hands one at a time:

```
effect Give = give : (Hand) → Unit

cycle = (a) ↦ then(give(a), cycle(answer(a)))
```

`cycle(rock)` gives rock, then the answer to rock, which is paper, then scissors, then rock again, forever. It is written as if nothing else were happening.

A handler can stop it at each hand, and give back the hand together with the rest:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves = (task) ↦ (u) ↦ handle task() with
                          (outcome) ↦ done
                        | (give(a), k) ↦ gave(a, k)
```

`moves` gives back a function waiting for `unit`. Given it, it starts the task and stops at the first hand. The rest, `k`, waits for `unit` too, the answer to `give`, and given it, goes on to the next hand and stops again. `k`, `k'` and `k''` are the rests after each hand:

```
moves(() ↦ cycle(rock))(unit)      comes to  gave(rock, k)
k(unit)                            comes to  gave(paper, k')
k'(unit)                           comes to  gave(scissors, k'')
```

`cycle` never ends, but it is carried out only as far as someone asks. This is a generator: it gives its items one at a time, each when asked for.

Two players can take turns. `turns` plays a number of rounds, asking each player in turn for a hand, and stops early if either has no more:

```
turns = (zero, first, second) ↦ empty | (succ(i), first, second) ↦ meet(i, first(unit), second(unit))
meet = (i, gave(a, first), gave(b, second)) ↦ cons(round(a, b), turns(i, first, second)) | (i, one,
      other) ↦ empty

rocks = () ↦ then(give(rock), rocks())

turns(succ(succ(succ(zero))), moves(() ↦ cycle(rock)), moves(rocks))
    comes to  cons(round(rock, rock), cons(round(paper, rock), cons(round(scissors, rock), empty)))
```

Each player is written as if it played alone. `turns` and `meet` hold both rests, and decide which goes on, and when.

So a handler can go on once, never, several times, or later. The same two forms, request and handler, say all four.

11. The outside

A request that no handler in the program answers is waiting, and whatever runs the program can answer it. Showing something on a screen, or reading what someone types, is such a request:

```
effect Console = show : (Hand) → Unit | read : () → Hand

game = () ↦ then(show(rock), beats(rock, read()))
```

`game()` waits at `show(rock)`. Whatever runs it shows rock on the screen and answers `unit`, which goes where the request was, and carrying out goes on. At `read()` it waits again, for a hand someone types. The outside works like one more handler, around the whole program, that always goes on once. Inside the language, a handler can stand in for the outside, which is how such a program is tested:

```
handle game() with
      (outcome) ↦ outcome
    | (show(a), k) ↦ k(unit)
    | (read(), k) ↦ k(scissors)

    comes to  true
```

Requests of different effects can be made together:

```
keep(() ↦ then(show(rock), both(round(paper, rock), round(rock, paper))), zero)
```

`keep`'s handler has no case for `show`, so `show(rock)` passes it by and goes outside. When the outside answers `unit`, the task goes on as before, and the whole comes to `pair(false, succ(zero))`.

A person can be a player, whose rule reads the next hand from the keyboard: `player((a) ↦ read())`. That is data, but not a `Player`, since its rule asks `Console`. A function type can also say what its function may ask:

```
(Hand) → Hand ! {Console}
```

This reads "from `Hand` to `Hand`, and may ask `Console`". A value is of this type when, applied to a hand, it never gets stuck, leaves no request unanswered but those of `Console`, and, with each of those answered as `Console` declares, ends, if it ends, in a hand. `answer` is one of these too: "may ask" includes not asking. So this `Player` holds a person as well as a rule:

```
Player = player((Hand) → Hand ! {Console})

next(player((a) ↦ read()), rock)
    comes to  read()
```

and waits there, for the person to choose.

On the map

Using `E` for the effects a function may ask:

```
A ::= (A, …) → B ! E       a function type, and the effects it may ask
E ::= {F(A, …), …}         effects, filled; F alone if it takes none

names    an effect is filled with as many types as it takes
ends     when the outside answers a waiting request, its answer goes where the request was,
         and carrying out goes on
types    a value is of type (A, …) → B ! E when, applied to values of types A, …, it never gets stuck,
         leaves no request unanswered but those of the effects in E, with inputs of the types they take,
         and, with each of those answered by a value of the type it is answered with, ends, if it ends,
         in a value of type B
```

12. The whole language

The letters: `e` an expression, `v` a value, `x` a placeholder, `c` a constructor, `n` a defined name, `p` a pattern, `d` a constructor as declared, `T` a type's name, `F` an effect's name, `r` a request, `h` a handler's case, `A` and `B` types, `X` a placeholder for a type, `E` the effects a function may ask. `e, …` means any number of expressions, even none, and likewise for the other letters. `| …` means more like the one before, as many as needed.

What we write: declarations and definitions, then one expression to carry out.

```
T(X, …) = d | …                        a type, with its constructors; T alone if it takes no types
d ::= c                                a constructor holding nothing
      c(A, …)                          a constructor holding contents of types A, …
effect F(X, …) = r : (A, …) → B | …    an effect, with its requests; F alone if it takes no types
n = (p, …) ↦ e | …                     a definition: n stands for a function, with its cases

parentheses only group
a body reaches as far as it can
```

Expressions, values, patterns

```
e ::= x                          a placeholder
      c                          a constructor
      c(e, …)                    data: a constructor with its contents
      n                          a defined name
      (p, …) ↦ e | …             a function written out
      e(e, …)                    an application
      r(e, …)                    a request
      handle e with h | …        a handler around e

v ::= c                          a constructor
      c(v, …)                    data: a constructor holding values
      n                          a defined name
      (p, …) ↦ e | …             a function written out

p ::= c                          matches only itself
      x                          a placeholder: matches anything
      c(p, …)                    matches data built by c, with contents matching p, …

h ::= (p) ↦ e                    for the value the expression inside comes to
      (r(p, …), x) ↦ e           for a request, and a placeholder for the rest
```

Types

```
A ::= T(A, …)                    a type, filled with types; T alone if it takes none
      X                          a placeholder for a type
      (A, …) → B ! E             a function type, and the effects it may ask; ! E left out when it lists
      none
E ::= {F(A, …), …}               effects, filled; F alone if it takes none
```

Names

```
each type and each effect is declared once; each constructor belongs to one type, and each request
  to one effect; each defined name is defined once
a declaration or a definition can use any type or defined name, its own included, even one declared or
  defined after it
in a declaration, every name in a type is a declared type, or one of the declaration's own placeholders
the placeholders of one declaration are all different, and none has the name of a declared type
a type or an effect is filled with as many types as it takes
in a pattern, a name is a constructor if it was declared as one, and otherwise a placeholder
the placeholders of one case are all different, and none has the name of anything declared or defined
a placeholder means something in the body of its own case, functions inside it included
in an expression, a name means the nearest placeholder of that name; with none, it must be declared or
      defined
the cases of one function all have the same number of patterns
a defined name written right before inputs is given as many as its function takes
a constructor is written with as many contents as it holds, in an expression or a pattern; one that
  holds nothing is written without parentheses
a request is written with as many inputs as it is declared with, in an expression or a handler's case
```

Steps

```
((p, …) ↦ e | …)(v, …)     becomes   the body of the first case whose patterns v, … match,
                                      with what each placeholder matched in its place
n(v, …)                    becomes   the same, with the cases of n's definition
handle v with h | …        becomes   the body of the first case for values that v matches,
                                      with what each placeholder matched in its place
handle e with h | …        becomes   the body of the first case for r whose patterns v, … match,
                                      with what they matched in place, and the rest in place of x,
                                      when the next thing to work out in e is a request r(v, …),
                                      and no handler in e around that request has a case for r

the rest is (x) ↦ handle e' with h | …, where e' is e with x where the request was,
  and x is a placeholder used nowhere in e or in the handler's cases
```

Where steps happen

```
in an application, first in what is applied, until it is a value; then in its inputs, from left to right
in data, in its contents, from left to right
in a request, in its inputs, from left to right
in a handler, in the expression inside it
nothing happens inside a function until it is applied
```

How carrying out ends

```
in a value
stuck: a value is where it doesn't fit, so something is left to work out but no step applies
never: carrying out goes on forever
waiting: the next thing to work out is a request that no handler around it has a case for; when the
  outside answers, its answer goes where the request was, and carrying out goes on
```

What a type means

```
a value is of type T(A, …)           when it is built by one of T's constructors, holding values of the
                                     types declared for its contents, with A, … in place of T's
      placeholders
a value is of type (A, …) → B ! E    when, applied to values of types A, …, it never gets stuck, leaves
      no
                                     request unanswered but those of the effects in E, with inputs of
      the
                                     types they take, and, with each of those answered by a value of the
                                     type it is answered with, ends, if it ends, in a value of type B

a type that holds itself: a value is of the type when following the definition, as far as it can be
  followed, never finds anything that does not fit
a type can hold itself in its contents, and in what a function gives back, but not in what a function
      takes
```

Nothing in carrying out uses types. That a value is of a type is a claim, and claims sit on top of the language.

The shape

	made by	used by
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs
effect	a request	a handler, which answers it

Six forms of expression for saying, two for doing, three forms of type, and a few rules for each. Everything else is a combination of them.

13. Reading another language

Many languages can be read through this map: some of its forms chosen, some hidden, and their own way of writing them. Python, for example:

Python	on the map
an `Enum` class	a type, listing its constructors
`True`, `False`, an `Enum`'s members	constructors
`def`	a definition
`lambda`, a `def` inside a function	a function written out, using the placeholders around it
`if`, `match`	cases
tuples, `dataclass`	constructors holding contents
`for`, `while`	a definition that uses itself
a variable	a memory at a place of its own
`score = score + 1`	`set(score, succ(look(score)))`, with its handler hidden
`b = a`, for a list `a`	two placeholders holding one place
`raise`, `try`	a request whose handler never goes on
`yield`	a request whose handler goes on later
`async`, `await`	requests whose rests a handler holds, and takes in turns
`print`, `input`	requests the outside answers
`TypeError`	what Python does where this language gets stuck
`Callable[[Hand], Hand]`	`(Hand) → Hand ! E`, with `E` unsaid: a Python function may ask anything
a type hint	a claim, checked by separate tools, on top of the language